

HelixOTA

HelixOTA

Actualizaciones universales y desacopladas por aire: cero bloqueos por diseño.

Resumen

Helix OTA es un sistema universal de actualizaciones por aire (OTA) profundamente desacoplado: un plano de control Go junto con agentes cliente por sistema operativo, diseñado para distribuir actualizaciones seguras y escalonadas de firmware/aplicaciones a flotas que van desde una sola placa hasta millones de dispositivos. Su primer objetivo es Android 15 en el Orange Pi 5 Max.

Descripción breve

Helix OTA es un sistema universal de actualizaciones por aire —un plano de control Go más agentes cliente por sistema operativo— diseñado para garantizar cero corrupción del sistema, validación de subidas y despliegues escalonados granulares. Su primer objetivo es Android 15 en el Orange Pi 5 Max, con adaptadores para Linux y Windows previstos.

Descripción detallada

Helix OTA es un sistema universal, genérico y profundamente desacoplado de actualizaciones por aire (OTA) construido bajo una promesa inquebrantable: una actualización nunca debe convertir un dispositivo funcional en un ladrillo. Está compuesto por un servidor **plano de control** Go, **SDKs/agentes** cliente por sistema operativo y un **panel de gestión**, y se ha diseñado desde cero para ser integrable en *cualquier* sistema operativo mediante adaptadores pluggables, en lugar de reconstruirse desde cero para cada plataforma. El primer objetivo de implementación es Android 15 (todas las variantes) en el Orange Pi 5 Max, donde el

pipeline de compilación genera imágenes de flasheo junto con un archivo .zip validado de OTA y los archivos de hash obligatorios, de modo que ningún artefacto llega a un dispositivo sin una huella verificable; Linux, Windows y otros sistemas operativos figuran en la hoja de ruta tras la misma interfaz de adaptador, a la espera solo de su adaptador —no de una reescritura—.

El diseño se estructura en torno a garantías estrictas definidas por el operador, tratadas como invariantes arquitectónicas no negociables: cero corrupción del sistema, validación obligatoria de cada artefacto antes de su despliegue, despliegue granular (todo a la vez o escalonado al 5/10/30...100% con control de pausa y avance), observabilidad completa de la flota y escalabilidad lineal desde una sola placa en un banco de pruebas hasta millones de dispositivos en campo. La arquitectura bloqueada combina actualizaciones nativas A/B de Android en el dispositivo — update_engine de AOSP con AVB/dm-verity y retroceso automático ante fallos de arranque— con un plano de control Go personalizado y desacoplado, de modo que la seguridad reside tanto en la ruta de arranque cercana al silicio *como* en el servidor, no en una sola capa frágil. Se mantienen deliberadamente dos interfaces extraíbles: una interfaz de adaptador de sistema operativo que garantiza verdadera universalidad, y una interfaz de motor de despliegue que hace que las campañas escalonadas sean agnósticas al sistema operativo. Todo el sistema se descompone en seis submódulos públicos ota-* versionados de forma independiente —bloques de construcción reutilizables en lugar de un monolito—.

Helix OTA se encuentra actualmente en una fase de especificación, investigación y desarrollo de cobertura de pruebas; el repositorio alberga el corpus de diseño autoritativo, el pipeline de exportación de documentación y el andamiaje de submódulos, y es explícito —según su gobernanza anti-engaño— que aún no existen un servidor y un agente de producción terminados. Lo que se distribuye hoy es el plano y su andamiaje, etiquetado con honestidad como tal.

Por qué lo creamos

OTA suele reinventarse para cada dispositivo y cada sistema operativo, y una mala actualización puede inutilizar toda una flota. Helix OTA se diseñó como un sistema de actualización universal y seguro por defecto, que cualquier SO puede adoptar mediante adaptadores, con garantías de reversión y validación integradas en la arquitectura desde el principio, en lugar de añadidas como parches.

Por qué es un cambio radical

Rechaza tratar “no inutilizar un dispositivo” y “desplegar de forma gradual y observable” como características de mejor esfuerzo que uno espera que resistan bajo carga: son invariantes arquitectónicos integrados tanto en la ruta de arranque como en el plano de control. Además, al convertir el motor de despliegue y la capa del SO en costuras intercambiables —en lugar de supuestos rígidos—, el mismo plano de control puede gestionar Android hoy y estar listo para operar con otros sistemas operativos mañana, simplemente añadiendo un adaptador: sin bifurcaciones, sin reescrituras, sin reinventar las garantías de seguridad en las que ya confías.

Qué lo hace innovador

- **Dos costuras extraíbles:** una costura de adaptador para el SO y un motor de despliegue agnóstico al sistema operativo, que convierten “universal” de un término de marketing en una propiedad estructural del código.
- **Seguridad en profundidad:** A/B nativo en el dispositivo (`update_engine`) + AVB/dm-verity + reversión automática ante fallos de arranque, superpuestos *sobre* la validación de artefactos en el servidor. Una actualización debe superar múltiples compuertas independientes antes de persistir.
- **Descomposición basada en catálogo y desacoplada:** seis submódulos ota-* reutilizables y versionados de forma independiente, que puedes consumir a la carta en lugar de tragarte un monolito.
- **Transporte primario HTTP/3 (QUIC)** con respaldo automático a HTTP/2 y compresión negociada Brotli/gzip: entrega moderna y de baja latencia que se degrada con elegancia en lugar de fallar.
- **Ingeniería anti-engaño:** el diseño y el estado se marcan explícitamente como en fase de especificación, y nunca se afirma que algo no construido esté ya implementado. La honestidad se impone como un valor de ingeniería de primer orden, no como una advertencia en letra pequeña.

Principales desafíos técnicos y cómo los resolvimos

- **Garantizar que una actualización defectuosa nunca inutilice un dispositivo:** la promesa más difícil en OTA. Se resolvió exigiendo A/B nativo en el dispositivo: `update_engine` escribe en la ranura inactiva mientras la activa sigue funcionando, AVB/dm-verity verifica criptográficamente la cadena de arranque, y si la nueva ranura falla al iniciar, el

dispositivo revierte automáticamente. Todo respaldado por una validación obligatoria de artefactos previa al despliegue, de modo que un paquete corrupto se detecta antes de que abandone el servidor.

- **Un sistema para múltiples sistemas operativos:** se resolvió evitando integrar supuestos de Android en el núcleo. Una costura de adaptador para el SO, conectable, aísla las especificidades de la plataforma, y un motor de despliegue agnóstico al SO mantiene la lógica de las campañas portátil. Ambos se mantienen como submódulos independientes, de modo que añadir un nuevo SO es una ampliación, nunca una cirugía al conjunto.
- **Despliegues por etapas y detenibles:** se resolvió con un motor de despliegue dedicado que opera con cohortes porcentuales, umbrales de éxito/error y controles explícitos de pausa/avance. Se diseñó deliberadamente libre de acoplamiento HTTP para que el mismo motor pueda gestionar campañas independientemente del transporte.

Pila tecnológica

- **Go + Gin** — seleccionados por su modelo de concurrencia y su reducido impacto en el despliegue; impulsan el plano de control, el motor de despliegue y los validadores de artefactos, exponiendo la superficie principal `/api/v1` de REST.
- **Kotlin/KMP** — elegidos para que el agente OTA en Android pueda compartir lógica entre distintos objetivos; gestionan el ciclo completo del dispositivo: sondeo, descarga, verificación, aplicación e informe.
- **HTTP/3 (QUIC) → HTTP/2** — QUIC se utiliza como transporte principal para ofrecer entrega de baja latencia y resiliente en enlaces móviles con pérdidas, con retroceso automático a HTTP/2 para que ningún dispositivo quede aislado; **Brotli/gzip** se negocia por solicitud para reducir el tamaño de las cargas.
- **PostgreSQL** — seleccionado por su integridad relacional en el registro de dispositivos, campañas y telemetría, donde la precisión del estado de la flota es más importante que la velocidad bruta de escritura.
- **MinIO / S3** — elegidos como almacenamiento de blobs de artefactos para que las imágenes de firmware de gran tamaño residan en almacenamiento de objetos estándar, desacoplado de la capa relacional.
- **AOSP update_engine + AVB/dm-verity + boot_control** — seleccionados porque reutilizar la maquinaria de Virtual A/B y arranque verificado de Android, probada en batalla, es más seguro que crear un actualizador personalizado; se emplean para gestionar los intercambios de ranuras y la verificación criptográfica del arranque en el dispositivo.

- **React** — elegido para el panel de gestión, donde los operadores inician sesión, suben artefactos, controlan los despliegues y supervisan el estado de la flota en un solo lugar.
- **OpenTelemetry + Prometheus/Grafana** — seleccionados para una instrumentación neutral en cuanto a proveedores; se utilizan para hacer observable cada etapa de un despliegue en métricas y paneles, en lugar de basarse en suposiciones.

Estado y notas de transparencia

- **Estado: en desarrollo.** Según la política de transparencia del proyecto, **aún no existe un servidor ni un agente en producción funcional**; esta es una fase de especificación, investigación y desarrollo de cobertura de pruebas. El repositorio contiene el corpus de diseño oficial, la canalización de exportación de documentación y el andamiaje de submódulos.
- Los seis submódulos públicos reutilizables (ota-protocol, ota-artifact-validator, ota-rollout-engine, ota-update-engine-bridge, ota-android-agent, ota-telemetry-schema) se encuentran en github.com/HelixDevelopment/.
- Las cifras de cobertura de pruebas y latencia en el repositorio son el registro en curso del propio proyecto, no han sido confirmadas de forma independiente. Los números de cláusulas de HelixConstitution citados en el README NO ESTÁN VERIFICADOS.
- **Licencia: Apache-2.0.**

Nivel de prioridad: Helix principal.